COER UNIVERSITY

7th km, Roorkee-Haridwar Highway, Haridwar, Uttarakhand

Lab Sheet-10: Capstone Machine Learning Projects

MCA III Semester (Session 2026–2027)

Project 1: Loan Approval Prediction System

Objective: Develop a machine learning application to predict loan approval status based on customer applicant profiles and create an interactive user interface configuration.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
import joblib

# Create standard baseline dataset synthetic array
np.random.seed(42)
n_samples = 500
data = {
    'Income': np.random.randint(20000, 150000, n_samples),
    'LoanAmount': np.random.randint(5000, 300000, n_samples),
    'CreditHistory': np.random.choice([0, 1], n_samples, p=[0.2, 0.8]),
    'Education': np.random.choice(['Graduate', 'Not Graduate'], n_samples),
    'EmploymentStatus': np.random.choice(['Employed', 'Self-Employed', 'Unemployed
'], n_samples),
    'LoanApproved': np.random.choice([0, 1], n_samples)
}
df = pd.DataFrame(data)

# Preprocessing Pipelines
le = LabelEncoder()
df['Education'] = le.fit_transform(df['Education'])
df['EmploymentStatus'] = le.fit_transform(df['EmploymentStatus'])

X = df.drop('LoanApproved', axis=1)
y = df['LoanApproved']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st
ate=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Classification Engine
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
model.fit(X_train_scaled, y_train)

# Evaluation Framework
predictions = model.predict(X_test_scaled)
print(f"Accuracy Metric Score: {accuracy_score(y_test, predictions)}")
print(classification_report(y_test, predictions))

# Serialization Persistence
joblib.dump(model, 'generated/loan_model.pkl')
joblib.dump(scaler, 'generated/loan_scaler.pkl')
print("Model saved to artifacts storage.")
```

Streamlit Deployment Script UI Configuration

```
import streamlit as st
import numpy as np
import joblib

model = joblib.load('loan_model.pkl')
scaler = joblib.load('loan_scaler.pkl')

st.title("Loan Approval Prediction System")
st.write("Provide applicant metrics parameters configuration criteria to execute a
nalysis:")

income = st.number_input("Applicant Income ($)", min_value=0, value=50000)
loan_amount = st.number_input("Requested Loan Amount ($)", min_value=0, value=1500
00)
credit_hist = st.selectbox("Credit History Metric Profile Status", [1, 0])
edu = st.selectbox("Education Level", ["Graduate", "Not Graduate"])
emp = st.selectbox("Employment Profile Status", ["Employed", "Self-Employed", "Une
mployed"])

edu_encoded = 0 if edu == "Graduate" else 1
emp_encoded = 0 if emp == "Employed" else (1 if emp == "Self-Employed" else 2)

if st.button("Run Eligibility Evaluation Analysis"):
    features = np.array([[income, loan_amount, credit_hist, edu_encoded, emp_encod
ed]])
    features_scaled = scaler.transform(features)
    prediction = model.predict(features_scaled)

    if prediction[0] == 1:
        st.success("Application Verdict Decision Result: LOAN APPROVED")
    else:
        st.error("Application Verdict Decision Result: LOAN REJECTED")
```

Project 2: Disease Prediction System

Objective: Develop a machine learning classification baseline module comparing algorithms to diagnose clinical condition variants based on patient medical metrics data arrays.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Generate multi-class medical feature metric distribution
np.random.seed(42)
samples = 600
data_medical = {
    'Fever': np.random.choice([0, 1], samples, p=[0.4, 0.6]),
    'Cough': np.random.choice([0, 1], samples, p=[0.3, 0.7]),
    'Fatigue': np.random.choice([0, 1], samples, p=[0.2, 0.8]),
    'SystolicBP': np.random.randint(110, 160, samples),
    'Cholesterol': np.random.randint(150, 280, samples),
    'Disease_Label': np.random.choice(['Healthy', 'Influenza', 'Hypertension', 'Di
abetes'], samples)
}
df_med = pd.DataFrame(data_medical)

X_med = df_med.drop('Disease_Label', axis=1)
y_med = df_med['Disease_Label']

X_tr, X_te, y_tr, y_te = train_test_split(X_med, y_med, test_size=0.2, random_stat
e=42)

# Algorithm 1: Multinomial Logistic Regression Configuration
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_tr, y_tr)
lr_preds = lr_model.predict(X_te)
print(f"Logistic Regression Model Engine Accuracy: {accuracy_score(y_te, lr_preds)
}")

# Algorithm 2: Support Vector Machine (SVM) Classification Kernels
svm_model = SVC(kernel='rbf', C=1.0)
svm_model.fit(X_tr, y_tr)
svm_preds = svm_model.predict(X_te)
print(f"Support Vector Machine (SVM) Classifier Engine Accuracy: {accuracy_score(y
_te, svm_preds)}")

print("\n--- Performance Metrics Report (Logistic Regression) ---")
print(classification_report(y_te, lr_preds))
```